```
# Step 3: repeat the full-table test for all 12 cycles; check counts and whether
table symmetries carry the change pattern across.
import numpy as np, io, contextlib
ns={}
with contextlib.redirect_stdout(io.StringIO()):
exec(open('46_table_only_pairing.py').read().split("# (c)")[0],ns)
G,N,res,analyse,pairing,conjugator,A=[ns[k] for k in "G N res analyse pairing
conjugator A".split()]
def Jdelta(perm):
    Pm,splits=analyse(perm)
    two=[k for k,v in splits.items() if len(v)==2]
    part,inner,outer,*_=pairing(Pm,two[0][0])
    J=np.arange(N)
    for i,o in part.items(): J[i]=o; J[o]=i
    D=G[np.ix_(J,J)]-G
    def ch(S,T):
        g=G[np.ix_(S,T)].flatten(); gj=G[np.ix_(J[S],J[T])].flatten(); d={}
        for a,b in zip(g,gj):
            if a!=b: d[(int(a),int(b))]=d.get((int(a),int(b)),0)+1
        return dict(sorted(d.items()))
    return two[0],J,D,inner,outer,ch(inner,inner),ch(outer,outer),ch(inner,outer)
base=Jdelta(res[0][1])
print("cycle 0: split",base[0],"| nonzero",int((base[2]!=0).sum()),"| inner-
inner",base[5],"| outer-outer",base[6],"| inner-outer",base[7])
for seed in range(1,12):
    pk=res[seed][1]; out=Jdelta(pk)
    s=conjugator(res[0][1],pk); S=np.array([s[r] for r in range(N)])
    sameJ=(out[1][S]==S[base[1]]).all()                        # J_k = s J_0 s^-1
    sameD=(out[2][np.ix_(S,S)]==base[2]).all()                 # Delta_k carried from
Delta_0 by s
    print(f"cycle {seed:2d}: split {out[0]} | nonzero {int((out[2]!=0).sum())} |
counts equal to cycle 0: {out[5:]==base[5:]} "
          f"| s carries J_0 onto J_{seed}: {sameJ} | s carries the whole change
pattern: {sameD} | changed entries shared with cycle 0: {int(((out[2]!=0)&
(base[2]!=0)).sum())}")
```